

Hochschule Ravensburg-Weingarten
University of Applied Sciences



Specification of

RISC-V 64-Bit Single-Instruction Microcontroller

Author: Yash Balu Sangale

Matriculation No.: 29346383

Study Program: M.Eng. Electrical Engineering and Embedded
Systems

Faculty of Electrical Engineering and Computer Science

Supervisor: Prof. Dr.-Ing. A. Siggelkow

Revision: 1.0

Date: January 22, 2026

Contents

1	Introduction	1
1.1	Purpose of Document	1
1.2	Scope	1
1.3	Reference Documents	1
2	Product Overview	2
2.1	Top Level Description	2
2.1.1	Introduction	3
2.1.2	Key Features	3
2.2	Architecture Concepts: Overview	3
2.2.1	Functional Block Diagram	3
2.2.2	Package	4
2.3	System View	4
2.3.1	Application System Description and Use Cases	4
2.3.2	System Description	4
2.3.3	System Integration and Application Circuit	5
2.3.4	Compliances	5
2.4	Architecture Concepts Overview	5
2.4.1	Technology	5
2.4.2	System Memory Concept	6
2.4.3	Software Architecture	6
2.4.4	Interprocessor Communication Concept	7
2.5	Functional Block Overview	7
2.6	Pin List	8
3	Architecture Concepts	9
3.0.1	Technology	9
3.0.2	Central Processing Unit (CPU) Concept	10
3.0.3	Bus Concept	11
3.0.4	Memory Concept	11
3.0.5	Interrupt Concept	12
3.0.6	Clock Concept	13
3.0.7	System Control Concept	13
3.0.8	Register Concept	13
3.0.9	Power Concept	14
3.0.10	System Protection and Security Concept	14
3.0.11	Debug Concept	14

4	Description of the Design Elements	15
4.1	CPU Core	15
4.1.1	HW Interfaces to Other Blocks	15
4.1.2	Bus Interface	16
4.1.3	Registers	16
4.1.4	Functional Overview	16
4.1.5	Structural Overview	17
4.1.6	Interrupts	17
4.1.7	Clock and Reset	17
4.1.8	Voltage Class	17
5	Toolchain	18
5.1	Introduction of the Toolchain	18
5.2	Technologies Used	18
5.2.1	Build and Automation Technologies	19
5.2.2	Simulation and Verification Tools	19
5.2.3	Development and Integration Tools	19
5.3	Usage of the Toolchain	19
5.3.1	Terminal-Based Usage	19
5.3.2	VS Code Extension Usage	20
5.4	Further Development of the Toolchain	21
5.4.1	CMake Enhancements	21
5.4.2	VS Code Extension Enhancements	21
6	Test And Debug	22
7	Validation and Test	23
8	Electrical Characteristics	24
8.1	Operating Conditions	24

List of Figures

2.1	Top Level Block Diagram of RISC-V 64-bit Integer Base ISA (RV64I) System-on-Chip	2
4.1	CPU Core interface diagram showing I-Bus, D-Bus, and Wishbone connections to system blocks.	16
4.2	CPU bus architecture — direct Harvard Architecture-style connections to Instruction Memory (I-MEM)/Data Memory (D-MEM) and Wishbone master for peripherals.	16
4.3	CPU Core internal structure showing Instruction Decoder, Arithmetic Logic Unit (ALU), Program Counter, and Register File.	17
4.4	Simplified Harvard Architecture architecture connection of CPU to I-MEM, D-MEM, and Wishbone peripherals.	17

List of Tables

1.1	History / Revision – Chapter 1	1
2.1	Revision History – Chapter 2	2
3.1	Revision History – Chapter 3	9
4.1	Revision History – Chapter 4	15
4.2	CPU Core Register Summary	16
5.1	History / Revision – Chapter 5	18
6.1	Revision History – Chapter 5	22
7.1	Revision History – Chapter 6	23
8.1	Revision History – Chapter 7	24
8.2	Operating Conditions	24

Glossary

Harvard Architecture A computer architecture with separate instruction and data memory paths. 2, 4, 6–8, 10, 11, 15–17

pipeline A series of processing stages that execute instructions in overlapping steps. 2, 4–7, 10, 11, 16, 17

System-on-Chip An integrated circuit containing processing cores, memory, and peripherals. 1–3, 6, 9, 15

Acronyms

ALU Arithmetic Logic Unit. 7, 10, 13, 16, 17

ASIC Application-Specific Integrated Circuit. 6, 9

CPU Central Processing Unit. ii, iii, 4–17

D-MEM Data Memory. 2, 5–8, 10–13, 15–17

FPGA Field-Programmable Gate Array. 4–7, 9

GPIO General Purpose Input/Output. 2–5, 7, 8, 11, 13–15

HDL Hardware Description Language. 9

I-MEM Instruction Memory. 2, 5–8, 10, 11, 13, 15–17

IRQ Interrupt Request or Interrupt Request Line. 3, 7, 8, 12–14, 17

ISA Instruction Set Architecture. 1, 2, 5, 7, 10

PC Program Counter. 10, 13, 16, 17

PWM Pulse Width Modulation. 2, 5, 8

QSPI Quad Serial Peripheral Interface. 2–5, 7, 8, 11, 13–15

RISC-V Reduced Instruction Set Computer - Five. 1, 2, 4, 6, 7, 9, 10, 16

RV64I RISC-V 64-bit Integer Base ISA. 2–5, 7, 10, 15, 16

SRAM Static Random-Access Memory. 9

UART Universal Asynchronous Receiver-Transmitter. 2–5, 7, 8, 11, 13–15

Chapter 1

Introduction

Table 1.1: History / Revision – Chapter 1

Version	Date	Author	Remarks
1.0	January 22, 2026	Y. Sangale	Initial template draft

1.1 Purpose of Document

This document defines the first specification of the Reduced Instruction Set Computer - Five (RISC-V) single-instruction-bit microcontroller, covering architectural, electrical, and functional aspects.

1.2 Scope

The specification outlines the design decisions, Instruction Set Architecture (ISA) subset, interfaces, and architectural details of the RISC-V System-on-Chip.

1.3 Reference Documents

- RISC-V Unprivileged ISA Specification, Volume I
- RISC-V Privileged Architecture Specification
- RWU “Specification Document Preparation Guidelines”

Chapter 2

Product Overview

Table 2.1: Revision History – Chapter 2

Version	Date	Author	Description
1.0	January 22, 2026	Y. Sangale	Initial Draft

2.1 Top Level Description

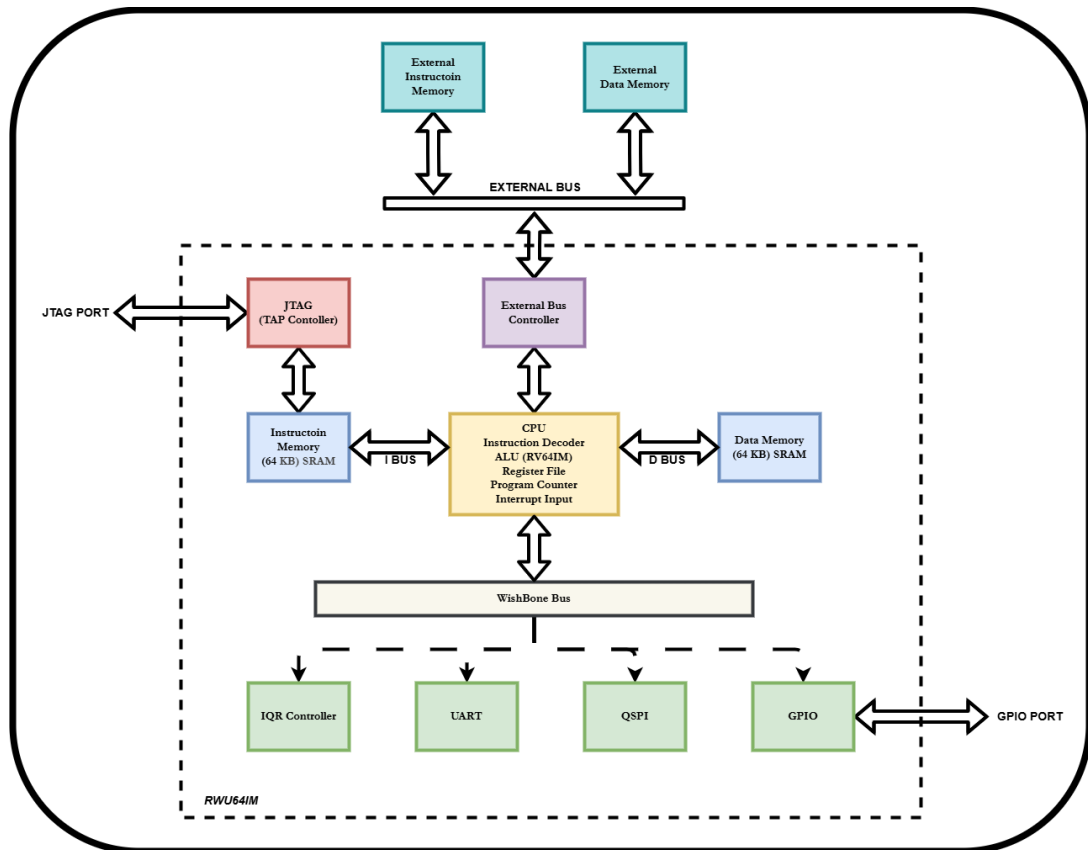


Figure 2.1: Top Level Block Diagram of RV64I System-on-Chip

2.1.1 Introduction

The designed microcontroller is a 64-bit RISC-V (RV64I) based single-core processor aimed at basic embedded applications. It operates at an internal clock frequency of 80 MHz and provides key interfaces such as General Purpose Input/Output (GPIO), Universal Asynchronous Receiver-Transmitter (UART), and Quad Serial Peripheral Interface (QSPI) for communication and JTAG for uploading Program on the controller. The architecture follows the Harvard Architecture model with separate instruction and data memories. A Wishbone-based interconnect provides standardized communication between the core and peripherals. The peripheral subsystem includes a configurable GPIO controller with interrupt capability, a UART for serial communication, and a QSPI interface for high-speed memory or peripheral access. The system integrates a clock generation block capable of supplying phase-shifted or divided clocks to peripherals, enabling dynamic clock configuration for Pulse Width Modulation (PWM) and serial interfaces. Programming and debugging are supported through a JTAG interface. The interrupt controller supports up to eight sources with programmable trigger modes (rising, falling, both, pulse). The design serves as the foundation for future extensions including cache, DMA, and additional interfaces.

2.1.2 Key Features

The RISC-V System-on-Chip microcontroller includes the following key features:

- RISC-V RV64I ISA
- Single-core, single-pipeline processor (scalar, in-order)
- Harvard Architecture architecture with separate I-MEM and D-MEM (64 KB each)
- 80 MHz internal main clock with peripheral clock dividers
- Wishbone bus architecture for peripheral connectivity
- 32×64 -bit general-purpose registers (x0–x31) and 64-bit program counter
- 8 configurable interrupt lines (Interrupt Request or Interrupt Request Line7:0 (IRQ)) with vector-based ISR dispatch
- Memory-mapped peripherals for modular extensibility
- UART, GPIO, QSPI interfaces
- JTAG for programming and on-chip debug access

2.2 Architecture Concepts: Overview

2.2.1 Functional Block Diagram

Functional blocks diagram will be included in the final version of this document.

Figure 2.1 shows the functional block diagram of the device. The architecture consists of a processing subsystem, peripheral subsystem, power management unit, and external interface controllers. Each block communicates via a shared system bus.

2.2.2 Package

Package type and pin configuration details will be provided in the final version of this document.

2.3 System View

2.3.1 Application System Description and Use Cases

The system is intended for use in low-power embedded applications such as IoT sensors, wearables, and smart home devices. A typical system setup includes power management, the microcontroller, and external communication modules such as BLE or Wi-Fi. The microcontroller is designed as a compact, easy-to-understand RV64I-based system suitable for educational purposes, Field-Programmable Gate Array (FPGA)-based prototyping, and simple embedded control applications. Its clean Harvard Architecture architecture, small peripheral set, and predictable interrupt behavior make it an ideal platform for teaching CPU pipeline concepts, memory systems, bus architectures, and peripheral interface design. Primary use cases include:

- Educational demonstrations of the RISC-V instruction set, pipeline execution, and memory-mapped I/O.
- FPGA-based prototyping of embedded systems involving UART, GPIO, or QSPI communication.
- Control-oriented applications such as sensor interfacing, simple automation systems, and custom peripherals.
- Research or coursework involving modification of the CPU pipeline, interrupt controller behavior, or clocking subsystem.

2.3.2 System Description

The system is organized as a Harvard Architecture-architecture microcontroller with separate instruction and data memories, unified through a Wishbone interconnect for all data accesses. The CPU implements the RV64I ISA and features a straightforward single-pipeline structure. The instruction memory (I-MEM) provides 64 KB of program space, while the data memory (D-MEM) offers an additional 64 KB region for stack, heap, and global variables.

All peripherals—including GPIO, UART, QSPI, the interrupt controller, and clock control module—are exposed via a memory-mapped address space connected through

the Wishbone bus. The CPU uses standard Wishbone read/write cycles for peripheral accesses, ensuring low design complexity and full visibility for educational study.

The system operates primarily on the 80 MHz main clock (CLK_MAIN). Optional phase-shifted or divided clocks are generated through the Peripheral Clock Generator (PCG) to support PWM generation, UART baud generation, or time-sensitive external interfaces.

2.3.3 System Integration and Application Circuit

For FPGA deployments, the system integrates cleanly with minimal external circuitry. Key considerations include:

- A stable 80 MHz clock source or PLL block configured within the FPGA fabric.
- External pins for UART, GPIO, QSPI, and JTAG made available on FPGA I/O banks.
- Proper pin constraints and I/O standards set according to the FPGA development board.
- Optional external QSPI flash memory interfaced through the QSPI lines for program storage or data logging.
- Decoupling capacitors on power rails and recommended pull-ups for JTAG lines depending on board design.

A typical application circuit includes CLK_MAIN input, reset circuitry, an external QSPI flash (optional), and standard UART TX/RX connectors.

2.3.4 Compliances

As the design is intended for FPGA-based educational and prototype systems, no formal compliance certification is mandated. For deployment in commercial hardware, considerations should include:

- Electromagnetic compatibility (EMC) guidelines for digital systems.
- ESD protection on external GPIO, UART, and QSPI pins.
- Compliance with RoHS and REACH directives for manufacturing.
- Ensuring JTAG and debug interfaces meet IEEE 1149.1 standards.

The modular design and clean clocking structure facilitate compliance with common embedded hardware regulations if needed.

2.4 Architecture Concepts Overview

2.4.1 Technology

The microcontroller is intended for implementation in a modern 90 nm CMOS technology node. This process offers high logic density, low leakage, and excel-

lent performance-per-watt characteristics, making it suitable for compact embedded microcontrollers and System-on-Chip-class designs. The design targets FPGA prototyping initially, with plans for Application-Specific Integrated Circuit (ASIC) fabrication in the future.

2.4.2 System Memory Concept

The microcontroller follows a Harvard Architecture architecture in which the Instruction Memory (I-MEM), Data Memory (D-MEM), and Register File are all directly connected to the CPU core without passing through the Wishbone bus. This separation enables parallel instruction fetch and data access, avoids structural hazards, and keeps the pipeline implementation simple and predictable.

Instruction Memory (I-MEM) is a 64KB block mapped at address range 0x0000 0000–0x0000 FFFF. It is accessed exclusively by the instruction fetch stage of the CPU via a dedicated instruction interface. Access is read-only from the CPU perspective, allowing efficient fetch timing and minimizing critical path delays.

Data Memory (D-MEM) is a separate 64KB block mapped at address range 0x0001 0000–0x0001 FFFF. The CPU accesses D-MEM through a dedicated data interface that supports load and store operations without involving any bus arbitration. Since D-MEM is not part of the Wishbone interconnect, data operations occur with deterministic timing independent of peripheral traffic.

The Register File consists of 32 general-purpose 64-bit registers (x0–x31) and is implemented inside the CPU core. It provides two read ports and one write port to support single-issue pipeline operation. The Register File is not memory-mapped and cannot be accessed by peripherals or external masters.

Peripheral Memory Space is accessed through the Wishbone bus and is distinct from both I-MEM and D-MEM. Only load/store instructions targeting the high address region (starting at 0x1000 0000) generate Wishbone transactions. This clean separation ensures that:

- program execution (I-MEM) never competes with data accesses,
- core data accesses (D-MEM) are unaffected by peripheral latency,
- Wishbone traffic originates only from memory-mapped I/O operations.

2.4.3 Software Architecture

The software architecture of the microcontroller is based on a conventional RISC-V bare-metal development flow. The hardware is implemented and verified using SystemVerilog, while Vivado is used as the primary environment for simulation, synthesis, and FPGA deployment.

Application development is performed using a RISC-V GCC toolchain configured for the RV64I ISA. Programs are written in C or assembly and compiled into machine code through the following steps:

1. C source files are compiled by the RISC-V compiler into assembly.

2. The assembler and linker generate a flat binary image targeting the I-MEM address map.
3. A custom programming tool converts the binary into a JTAG-compatible format.
4. Using the JTAG interface, the tool loads the program directly into the Instruction Memory implemented on the FPGA.

The microcontroller operates without an operating system, using a simple bare-metal startup sequence that initializes the stack pointer, configures peripherals, and optionally enables interrupts. All peripherals (GPIO, UART, QSPI, clock controller, and IRQ controller) are controlled using memory-mapped register access within the high-address I/O region. The interrupt architecture supports hardware-vector entry, simplifying handler development and reducing interrupt latency.

2.4.4 Interprocessor Communication Concept

The current microcontroller is a single-core RV64I design and therefore does not implement any interprocessor communication (IPC) mechanisms. All computation, memory access, and interrupt handling occur within the single CPU pipeline.

2.5 Functional Block Overview

The microcontroller consists of a compact set of functional modules designed around a clean Harvard Architecture architecture and a Wishbone-based peripheral subsystem. The major functional blocks include:

- **CPU Core (RV64I):** A single-issue scalar pipeline implementing the RV64I instruction set. Includes the register file, ALU, multiplication/division unit, instruction decoder, control logic, and interrupt handling interface.
- **Instruction Memory (I-MEM):** A 64KB program memory directly connected to the CPU fetch stage. Stores executable code and reset vectors.
- **Data Memory (D-MEM):** A 64KB data RAM directly accessed by the CPU load/store unit, supporting low-latency memory operations independent of peripheral traffic.
- **Wishbone Interconnect:** A simple single-master bus used by the CPU to access all memory-mapped peripherals. Ensures modular peripheral integration and predictable transaction behavior.
- **GPIO Controller:** Provides general-purpose digital I/O pins with configurable direction and interrupt capability. Supports optional PWM via peripheral clock division.
- **UART Controller:** Implements asynchronous serial communication with programmable baud rate and interrupt-driven transmit/receive support.
- **QSPI Controller:** Provides quad-SPI communication for external flash or high-speed serial devices.

- **Interrupt Controller:** Supports eight individually configurable external interrupt lines with level, edge, and pulse triggering. Generates hardware vector addresses to accelerate ISR entry.
- **Peripheral Clock Generator (PCG):** Generates divided and phase-adjusted clock signals derived from the 80 MHz main clock for use by UART, GPIO timers, QSPI, and other peripherals.
- **JTAG Interface:** Enables programming and debugging through a standard IEEE 1149.1-compatible interface.

Each module is memory-mapped in the high-address region and integrated through the Wishbone bus, while the CPU, Register File, I-MEM, and D-MEM remain directly connected as part of the Harvard Architecture core architecture.

2.6 Pin List

External pins for the microcontroller include:

- **CLK_MAIN** – Primary 80 MHz system clock input.
- **RESET_N** – System reset, active low.
- **GPIOx** – General-purpose digital input/output pins; each configurable for direction and interrupt triggering.
- **UART_TX, UART_RX** – UART communication pins.
- **QSPI_SCLK, QSPI_CS, QSPI_IO[3:0]** – Quad-SPI interface for external flash memory.
- **IRQ7:0** – External interrupt request inputs.
- **JTAG_TCK, JTAG_TMS, JTAG_TDI, JTAG_TDO, JTAG_TRST**
 - Debug/programming pins.

Pin configuration, multiplexing options, and detailed signal descriptions will be provided in the final revision of this document.

Chapter 3

Architecture Concepts

Table 3.1: Revision History – Chapter 3

Version	Date	Author	Description
1.0	January 22, 2026	Y. Sangale	Initial Draft

This section defines the fundamental architectural principles of the 64-bit RISC-V single-core microcontroller. It provides a high-level technical description of the CPU core, memory subsystem, bus structure, interrupt design, clocking strategy, and register architecture.

Concepts related to power management, security, and debug will be fully defined in the final specification revision.

3.0.1 Technology

The microcontroller is conceptually targeted for implementation in a modern **90 nm CMOS technology node**. This node provides a balanced compromise between integration density, power efficiency, and manufacturing cost, making it suitable for compact embedded microcontrollers and low-complexity System-on-Chip designs.

The 90 nm node enables:

- Higher operating frequencies compared to older planar CMOS nodes.
- Reduced dynamic and static power consumption.
- Dense Static Random-Access Memory (SRAM) bitcells allowing larger memories in a small silicon footprint.

Although the conceptual design and functional model are developed and validated using synthesizable Hardware Description Language (HDL) on FPGA platforms, the microarchitecture, clocking structures, and memory system were designed with 90 nm ASIC fabrication constraints in mind. This includes fully synchronous logic, controlled critical paths, and modular blocks suitable for automated place-and-route flows.

3.0.2 CPU Concept

The CPU core is a compact, single instruction RV64I processor optimized for determinism and simplicity.

- **Pipeline:** A simple scalar pipeline (Fetch–Decode–Execute–Writeback) without speculation or out-of-order features.
- **ISA Support:** Fully compliant with RV64I + M-extension (integer multiply/divide).
- **Datapath Width:** 64-bit ALU, registers, and internal buses.
- **Register File:** 32×64 -bit general-purpose registers (x0–x31), with direct CPU access (no bus).
- **Memory Access Model:** Harvard Architecture architecture — separate I-MEM and D-MEM buses.
 - Dedicated instruction interface to I-MEM.
 - Dedicated data interface to D-MEM.
 - Memory-mapped I/O via Wishbone for all peripherals.
- **Program Counter:** 64-bit Program Counter (PC) with generation logic (PC +4, branch target, jump target).
- **Interrupt Integration:** Receives a single aggregated interrupt line from the Interrupt Controller and vectoring is performed as per RISC-V rules.
- **Pipeline:** Single-stage or simplified multi-stage pipeline (Fetch → Decode → Execute → Writeback).
- **Data Width:** 64-bit ALU, register file, and datapath.
- **Instruction Execution:** One instruction executed at a time; no out-of-order execution.
- **ISASupport:** RV64I with full support for:
 - 64-bit integer arithmetic and logical operations,
 - Load/store with byte, half-word, word, and double-word support,
 - Multiply/divide (M-extension),
 - Branch and jump instructions.
- **Interrupt Input:** Single external interrupt line fed by the Interrupt Controller.
- **Memory Access:** Harvard Architecture architecture — separate I-MEM and D-MEM buses.

3.0.3 Bus Concept

The microcontroller employs a simple and predictable communication architecture centered around a **single-master bus topology**. The CPU core is the only bus master and is responsible for initiating all data and peripheral access transactions. This simplifies arbitration, reduces latency, and provides deterministic behavior suitable for education, prototyping, and low-complexity embedded applications.

The system uses two categories of interconnects:

- **Direct CPU Connections** The following modules are connected directly to the CPU datapath and do not use the system bus:
 - **Instruction Memory (I-MEM):** Accessed through a dedicated instruction fetch interface.
 - **Data Memory (D-MEM):** Connected through a dedicated data access path for load/store instructions.
 - **Register File (RF):** Integrated directly into the CPU pipeline.

These direct connections enable single-cycle access to critical datapath components and support a clean Harvard Architecture architectural split.

- **Wishbone-Based Peripheral Bus** All other memory-mapped peripherals are connected using a **Wishbone-compliant** slave interface. This bus is used for:
 - General Purpose I/O (GPIO),
 - Universal Asynchronous Receiver/Transmitter (UART),
 - Quad-SPI Master (QSPI),
 - Interrupt Controller,

The Wishbone bus provides standardized signaling for address, data, strobes, and acknowledgments, enabling:

- Modular and plug-and-play peripheral integration,
- Predictable, single-master timing behavior,
- Straightforward extension with new peripherals,
- Clean separation between CPU datapath and peripheral logic.

3.0.4 Memory Concept

The memory subsystem is based on a clean Harvard Architecture architecture with separated instruction and data paths.

- **Instruction Memory (I-MEM):**
 - 64 KB read-only memory for program storage.
 - Dedicated CPU instruction fetch path (non-Wishbone).
 - High-performance single-cycle access for most operations.

- **Data Memory (D-MEM):**
 - 64 KB data RAM for stack, heap, and program data.
 - Direct CPU data access (non-Wishbone).
 - Fully synchronous with CPU core.
- **Memory-Mapped I/O:**
 - Peripherals mapped in the 0x1000_0000 address region.
 - Accessed exclusively via the Wishbone peripheral bus.
- **Extensibility:**
 - Optional I-Cache and D-Cache may be added in future without changing the CPU-bus interface.

3.0.5 Interrupt Concept

The interrupt subsystem is based on a configurable multi-source interrupt controller supplying a single interrupt input to the CPU.

- **IRQ Lines:** 8 independent inputs (IRQ7:0) with per-line configuration.
- **Trigger Types:**
 - Level-high
 - Rising edge
 - Falling edge
 - Both edges
 - Pulse/one-shot
- **Edge Detection:** Implemented using per-line previous-state registers.
- **Control Registers:** Mask, type-select, and pending-status registers accessible via Wishbone.
- **Vector Computation:**
$$\text{ISR Vector} = \text{BASE_ADDR} + (\text{IRQ_ID} \times 0x10)$$
- **Priority Model:** Fixed priority (IRQ0 highest) for the initial revision.
- **CPU Interface:** Outputs a single interrupt line to the CPU's machine-mode interrupt system.

3.0.6 Clock Concept

The microcontroller uses an 80 MHz main system clock with optional peripheral clock division.

- **Core Clock:** 80 MHz clock feeding CPU, I-MEM, and D-MEM.
- **Peripheral Clocks:** Generated via a programmable divider block:
 - Divide-by-2, /4, /8, etc.
 - Per-peripheral selectable clock domains.
- **Clock Control Registers:** Accessible via Wishbone.
- **Clock Domain Crossing:** Standard two-flip-flop synchronizers used where required (e.g., UART, QSPI).

3.0.7 System Control Concept

System control is centered around a simple and robust global reset architecture. The microcontroller implements an **external GPIO-based reset input**, which provides a direct and intuitive mechanism to reinitialize the entire system.

- **External Reset Input:** A dedicated GPIO pin is configured as a reset input. When driven **low**, it asserts a system-wide synchronous reset.
- **Reset Behavior:**
 - All CPU state (PC, registers, CSRs) is reset to defined initial values.
 - Instruction Memory and Data Memory interfaces are reset.
 - All Wishbone peripherals (GPIO, UART, QSPI, IRQ controller, Clock Control, JTAG region) return to default configuration.
 - All internal state machines return to IDLE.
- **Synchronization:** The external reset signal passes through a two-stage synchronizer to ensure safe integration into the main clock domain.
- **Reset Release:** When the external GPIO reset input returns high, the system exits reset synchronously at the next active clock edge.

This minimal reset architecture provides deterministic system initialization and simplifies external control during prototyping and debugging.

3.0.8 Register Concept

The Register Concept defines the structure and behavior of all **memory-mapped peripheral registers** accessible through the Wishbone bus. These registers are used to configure, control, and monitor the operation of GPIO, UART, QSPI, the Interrupt Controller, the Clock Control Unit, and other on-chip peripherals.

- **Scope:** Only peripheral registers are included. CPU-internal structures such as the Register File and CSRs are part of the CPU microarchitecture and are not memory-mapped.
- **Register Types:**
 - Control registers (enable, mode select, configuration)
 - Status registers (interrupt flags, error states, peripheral activity)
 - Data registers (UART TX/RX, QSPI data buffer, GPIO input/output)
 - Configuration registers (IRQ trigger type, GPIO direction, clock divisor)
- **Access Method:** All registers are accessed via standard 32-bit or 64-bit CPU load/store instructions through the Wishbone data bus.
- **Address Alignment:** All registers are aligned to 32-bit word boundaries.
- **Reset Behavior:** On system reset, all registers revert to defined default states to ensure predictable system startup.

3.0.9 Power Concept

Section to be completed in the final document revision.

3.0.10 System Protection and Security Concept

Section to be completed in the final document revision.

3.0.11 Debug Concept

Section to be completed in the final document revision.

Chapter 4

Description of the Design Elements

Table 4.1: Revision History – Chapter 4

Version	Date	Author	Description
1.0	January 22, 2026	Y. Sangale	Initial Draft

This Section will be completed later.

This chapter provides a detailed technical description of all major design elements composing the RV64I 64-bit single-core microcontroller. Each sub-block is presented in terms of hardware interfaces, bus connectivity, internal registers, and its functional and structural roles within the System-on-Chip. All diagrams and tables are placeholders for later detailed schematics.

4.1 CPU Core

4.1.1 HW Interfaces to Other Blocks

The CPU Core is the central processing unit of the system, implementing the RV64IM instruction set and acting as the single bus master in the microcontroller architecture. It connects to all major subsystems as follows:

- **Instruction Memory (I-MEM):** Connected directly via the **Instruction Bus (I-Bus)** for program fetch under the Harvard Architecture architecture model.
- **Data Memory (D-MEM):** Connected directly via the **Data Bus (D-Bus)** for load/store access.
- **Wishbone Bus:** Acts as the **bus master** interfacing to all memory-mapped peripherals (GPIO, UART, QSPI, Interrupt Controller, and Clock Unit).
- **Interrupt Input:** Receives a single aggregated interrupt signal from the Vectored Interrupt Controller (VIC).
- **Clock and Reset:** Driven by the system clock from the Clock Generation Unit (CGU) and synchronized reset signal.

Figure 4.1: CPU Core interface diagram showing I-Bus, D-Bus, and Wishbone connections to system blocks.

4.1.2 Bus Interface

The CPU operates as a **Wishbone Master** for all peripheral transactions. It initiates read/write cycles with standard Wishbone control signals: `ADR_0`, `DAT_0`, `DAT_I`, `STB_0`, `ACK_I`, `CYC_0`, and `WE_0`. Instruction and data memories are accessed directly via the independent I-Bus and D-Bus, each 64-bit wide for full datapath utilization.

Figure 4.2: CPU bus architecture — direct Harvard Architecture-style connections to I-MEM/D-MEM and Wishbone master for peripherals.

4.1.3 Registers

The CPU core implements 32×64 -bit general-purpose registers (`x0-x31`) and a 64-bit program counter. Several internal control and status registers are included for interrupt and pipeline management.

Table 4.2: CPU Core Register Summary

Register	Description
PC	64-bit Program Counter for sequential and branch control
IR	Instruction Register holding current fetched instruction
SR	Status Register with control and flag bits
IBASE	Base address for interrupt vector table
MSTATUS	Machine Status Register (RV64I subset)

4.1.4 Functional Overview

The CPU executes instructions using a deterministic scalar pipeline that supports the RISC-V RV64IM base instruction set. Internally, it consists of the following major sub-modules:

- **Instruction Decoder:** Decodes opcodes fetched from I-MEM and generates control signals for ALU and memory.
- **Program Counter (PC):** Maintains the address of the current instruction and computes sequential or branch targets.
- **ALU:** Performs arithmetic, logical, shift, and comparison operations; supports integer multiply/divide.
- **Register File:** Provides two read ports and one write port for operand access and result storage.
- **Interrupt Input:** Interfaces with VIC to handle asynchronous interrupt requests.

Each instruction is executed in a simple in-order flow without speculation or pipeline hazards beyond single-cycle stalls.

Figure 4.3: CPU Core internal structure showing Instruction Decoder, ALU, Program Counter, and Register File.

4.1.5 Structural Overview

The CPU architecture follows a Harvard Architecture model with:

- Dedicated instruction path (I-Bus) for fetching 64-bit instructions.
- Dedicated data path (D-Bus) for memory and peripheral data transactions.
- Unified Wishbone interface for peripheral control and configuration.

The internal control path coordinates fetch, decode, execute, and memory operations. Instruction flow synchronization ensures deterministic timing suitable for embedded applications.

Figure 4.4: Simplified Harvard Architecture architecture connection of CPU to I-MEM, D-MEM, and Wishbone peripherals.

4.1.6 Interrupts

A single interrupt input line connects to the VIC. The CPU jumps to a vector address computed as:

$$\text{ISR Vector} = \text{IBASE} + (\text{IRQ_ID} \times 0x10)$$

Upon completion, control returns to the instruction following the interrupted instruction.

4.1.7 Clock and Reset

The CPU operates at an 80 MHz clock supplied by the CGU. Reset initializes the PC to 0x0000_0000, clears all pipeline stages, and resets the internal status registers.

4.1.8 Voltage Class

Core operates in the 3.3 V logic domain.

Chapter 5

Toolchain

Table 5.1: History / Revision – Chapter 5

Version	Date	Author	Remarks
1.0	January 22, 2026	Y. Sangale	Initial template draft

5.1 Introduction of the Toolchain

The development of a modern System-on-Chip (SoC) requires a robust, automated, and developer-friendly toolchain that seamlessly integrates hardware design, firmware development, simulation, testing, and documentation. For the **RV64I_RWU** project, a comprehensive toolchain was designed and extended to support the full development lifecycle of a 64-bit RISC-V processor core.

The toolchain is centered around a **modular CMake-based build system** and is complemented by a custom-developed **Visual Studio Code (VS Code) extension**. This combination enables both command-line-driven workflows for automation and graphical workflows for developer productivity. The primary goals of the toolchain are:

- Reproducible and cross-platform builds
- Automated firmware compilation and hardware simulation
- Scalable regression testing using CTest
- Reduced onboarding effort for new developers
- Tight integration between software, hardware, and verification flows

By abstracting complex build and simulation commands behind well-defined interfaces, the toolchain significantly improves usability while maintaining full transparency and flexibility for advanced users.

5.2 Technologies Used

The toolchain is built using industry-standard tools and technologies to ensure portability, maintainability, and long-term scalability.

5.2.1 Build and Automation Technologies

- **CMake (≥ 3.20):** Used as the primary build system generator. A modular CMake architecture enables auto-discovery of RTL, firmware, and test files while maintaining a clean separation of concerns.
- **CTest:** Integrated with CMake to provide automated unit, integration, firmware, and IP-level regression testing with support for parallel execution.
- **RISC-V GNU Toolchain:** The `riscv64-unknown-elf-gcc` compiler is used for compiling firmware written in assembly and C for the RV64I architecture.

5.2.2 Simulation and Verification Tools

- **Xilinx Vivado XSIM:** Used for RTL simulation, supporting both console-based execution for automation and GUI-based execution with waveform visualization.
- **TCL Scripts:** Employed to standardize waveform setup, simulation control, and batch execution in XSIM.

5.2.3 Development and Integration Tools

- **Visual Studio Code:** Serves as the primary development environment.
- **Node.js:** Used to develop a custom VS Code extension that provides an integrated user interface for the toolchain.
- **Git:** Used for version control and collaborative development.
- **Sphinx:** Used for generating HTML and PDF documentation directly from the project source.

5.3 Usage of the Toolchain

The RV64I toolchain supports two primary usage modes: a terminal-based workflow and an IDE-based workflow using the VS Code extension.

5.3.1 Terminal-Based Usage

The terminal workflow provides full control over the build and simulation process and is well suited for automation, scripting, and continuous integration environments.

Configuration and Build

```
cmake -S . -B build -G "MinGW Makefiles"
cmake --build build
```

This step configures the project, detects available tools, and builds all auto-discovered targets.

Firmware Compilation

```
cmake --build build --target fw_asm_instr06addi
```

Firmware targets are generated automatically based on the source files present in the firmware directory.

Simulation Execution

Console Mode:

```
xsim instr06addi_snapshot -runall
```

GUI Mode with Waveforms:

```
xsim instr06addi_snapshot --gui --tclbatch sim/config/xsim_cfg2.tcl
```

Regression Testing

```
ctest --test-dir build -j
```

CTest enables parallel execution of unit, integration, firmware, and IP-level tests.

5.3.2 VS Code Extension Usage

To improve usability and reduce the cognitive load of complex command sequences, a custom VS Code extension named **rwu-rv64i-toolchain** was developed.

Setup Wizard

The extension provides an interactive setup wizard that:

- Detects installed tools (CMake, RISC-V GCC, XSIM, Git)
- Reports missing components
- Allows manual configuration of executable paths

This significantly simplifies first-time setup, especially on Windows systems.

Build and Simulation Commands

The extension exposes high-level commands via the Command Palette, including:

- Configure and Build All
- Build Specific Target (compile only)
- Run Simulation (console mode)
- Run Simulation (GUI with waveforms)

Target names are automatically inferred from the currently opened file, reducing manual input errors.

Regression Testing Support

The extension integrates CTest commands, enabling:

- Execution of specific tests
- Full library regression testing
- Parallel test execution

All test results are displayed directly in the VS Code terminal for immediate feedback.

5.4 Further Development of the Toolchain

While the current toolchain is fully functional, several extensions and improvements are planned.

5.4.1 CMake Enhancements

Future improvements to the CMake-based build system include:

- Support for multiple simulators beyond XSIM
- Improved dependency visualization and reporting
- Enhanced cross-compilation support for additional RISC-V variants
- Integration with continuous integration (CI) pipelines

5.4.2 VS Code Extension Enhancements

Planned enhancements to the VS Code extension include:

- Graphical dashboards for build and test status
- Integrated waveform viewers
- Per-target configuration profiles
- Deeper integration with documentation and architecture diagrams

These improvements aim to further reduce development time and improve the overall developer experience.

Chapter 6

Test And Debug

Table 6.1: Revision History – Chapter 5

Version	Date	Author	Description
1.0	January 22, 2026	Y. Sangale	Initial Draft

This Section will be completed later.

Describe how the SoC will be tested or simulated for functionality. Include details on testbenches, simulation tools, and debugging interfaces.

Chapter 7

Validation and Test

Table 7.1: Revision History – Chapter 6

Version	Date	Author	Description
1.0	January 22, 2026	Y. Sangale	Initial Draft

This Section will be completed later.

List the memory map and all registers with its addresses and reset values in tabluar format. This includes all peripheral registers and CPU control and status registers (CSRs).

Chapter 8

Electrical Characteristics

Table 8.1: Revision History – Chapter 7

Version	Date	Author	Description
1.0	January 22, 2026	Y. Sangale	Initial Draft

This Section will be completed later.

8.1 Operating Conditions

Table 8.2: Operating Conditions

Parameter	Typical	Unit
Supply Voltage	3.3	V
Clock Frequency	80	MHz
Operating Temp Range	-40 to +85	°C

Bibliography